ITP API - Interpersonal Transaction Protocol API

ITP API

Interpersonal Transaction Protocol API

1. Overview

The **Interpersonal Transaction Protocol API**, abbreviated as **ITP API**, enables JavaScript web applications to interact with simulation models hosted by a Python server.

An interaction consists of two agents. Each agent can be either:

- A formal model
- A human participant

The following dyad configurations are supported:

- Formal model and formal model
- Human participant and formal model
- Formal model and human participant
- Human participant and human participant

The server is responsible for:

- Providing information about available simulation models
- Creating and managing interaction sessions
- Registering human participants
- Executing formal model agents
- Managing interaction turns
- Validating perceptual and behavioral state vectors
- Temporarily or persistently storing interaction data
- Providing perceptions through polling
- Accepting behavioral responses
- Terminating interactions

For human-to-human interactions, a six-digit invitation code allows a second instance of the JavaScript application to join the interaction and register the second human agent.

Polling is the communication mechanism between JavaScript applications and the server.

---

2. General Conventions

2.1 Base URL

```
/api/v1
```

2.2 Content Type

Requests and responses use JSON unless otherwise specified.

```
Content-Type: application/json
```

2.3 Authentication

When an interaction is created, the server returns one interaction-scoped token.

Protected endpoints use this token as a bearer token:

```
Authorization: Bearer <interactionToken>
```

For the initial API version, the same interaction token may be used by all clients participating in the interaction.

The second JavaScript application receives the interaction token only after successfully joining the interaction with the invitation code.

Tokens must never be transmitted as URL parameters.

2.4 Identifiers

The following resources should use globally unique, non-sequential identifiers such as UUIDs:

- Session IDs
- Agent IDs
- Action IDs
- Perception IDs
- Participant IDs

2.5 Timestamps

All timestamps use ISO 8601 in UTC.

```
2026-07-31T08:45:00Z
```

2.6 Naming Conventions

JSON property names and status values use `camelCase`.

API paths use lowercase plural resource names.

---

3. Interaction States

An interaction can have one of the following states:

```
waitingForRegistration
ready
running
completed
terminated
expired
failed
```

The usual lifecycle is:

```
waitingForRegistration
          |
          v
        ready
          |
          v
       running
          |
          v
completed | terminated | failed
```

If both agents are formal models, no participant registration is required. The interaction can immediately transition to `ready` or `running`.

If at least one agent is human, the interaction remains in `waitingForRegistration` until all required human agents have been registered.

The server starts the interaction automatically when all required registrations have been completed.

---

4. Model Endpoints

4.1 List Available Models

```
GET /api/v1/models
```

Returns the simulation models that are currently available for new interactions.

Response

```
200 OK
```


```
{
  "models": [
    {
      "id": "social-exchange",
      "version": "2.1.0",
      "name": "Social Exchange Model",
      "description": "A turn-based dyadic social interaction model.",
      "status": "available",
      "supportedAgentTypes": [
        "human",
        "model"
      ],
      "links": {
        "self": "/api/v1/models/social-exchange"
      }
    }
  ]
}
```


Possible model statuses are:

```
available
maintenance
deprecated
unavailable
```

Only models with the status `available` can be used to create new interactions.

4.2 Retrieve a Model Definition

```
GET /api/v1/models/{modelId}
```

Returns the model metadata and its state-space definition.

The perceptual state space and behavioral state space are identical. Therefore, the model exposes one shared JSON Schema named `stateSpaceSchema`.

The schema is used to validate both:

- Experience vectors returned by the perception endpoint
- Behavioral vectors submitted to the action endpoint

Response

```
200 OK
```

```
{
  "id": "social-exchange",
  "version": "2.1.0",
  "name": "Social Exchange Model",
  "description": "A turn-based dyadic social interaction model.",
  "interactionMode": "turnBased",
  "stateSpaceSchema": {
    "$comment": "Insert the complete JSON Schema from the provided state-space schema file here."
  },
  "constraints": {
    "minimumTurns": 1,
    "maximumTurns": 1000,
    "supportsPersistence": true
  }
}
```

The `stateSpaceSchema` property must contain the complete JSON Schema supplied for the behavioral state space.

The placeholder shown above is not part of the production API and must be replaced with the complete schema from the source file.

The server must use the exact same schema when validating perceptions and behaviors.

Conceptually:

```
schema(experience) = schema(behavior) = stateSpaceSchema
```

---

5. Create an Interaction

```
POST /api/v1/interactions
```

Creates a new interaction session.

5.1 Request

```
{
  "modelId": "social-exchange",
  "modelVersion": "2.1.0",
  "numberOfTurns": 20,
  "persistence": {
    "enabled": true
  },
  "agents": {
    "A": {
      "type": "human",
      "displayName": "Participant A"
    },
    "B": {
      "type": "human",
      "displayName": "Participant B"
    }
  },
```

```
  "clientRequestId": "create-interaction-7d14d66f"
}
```

5.2 Request Properties

`modelId`

Identifies the simulation model used for the interaction.

`modelVersion`

Identifies the exact model version.

Specifying the version is recommended so that the interaction remains reproducible if a newer model version is introduced.

`numberOfTurns`

Specifies the maximum number of interaction turns.

The value must comply with the model constraints returned by:

```
GET /api/v1/models/{modelId}
```

`persistence.enabled`

Determines whether the interaction data should remain available after the interaction has ended.

When set to `false`, the server should retain data only for as long as operationally necessary.

`agents`

Defines agents `A` and `B`.

Supported agent types are:

```
human
model
```

Formal model agents can receive an optional model-specific `configuration` object.

Example:

```
{
  "type": "model",
  "configuration": {
    "strategy": "adaptive",
    "randomSeed": 12345
  }
}
```

`clientRequestId`

A client-generated idempotency identifier.

If the client repeats the request with the same `clientRequestId`, the server should return the previously created interaction rather than create a duplicate.

5.3 Response for a Human-to-Human Interaction

```
201 Created
```


```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "interactionToken": "interaction-scoped-access-token",
  "invitation": {
    "code": "483921",
    "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
    "agentRole": "B",
    "status": "pending",
    "expiresAt": "2026-07-31T09:15:00Z"
  },
  "status": "waitingForRegistration",
  "model": {
    "id": "social-exchange",
    "version": "2.1.0"
  },
  "numberOfTurns": 20,
  "currentTurn": 0,
  "persistence": {
    "enabled": true
  },
  "agents": {
    "A": {
      "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
      "role": "A",
      "type": "human",
      "registrationStatus": "pending"
    },
    "B": {
      "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
      "role": "B",
      "type": "human",
      "registrationStatus": "pending"
    }
  },
  "createdAt": "2026-07-31T08:45:00Z",
  "links": {
    "self": "/api/v1/interactions/2fd2bfb0-f193-4f34-a16d-a1a5936bc734"
  }
}
```


The invitation is bound to agent `B`, which represents the second human participant.

5.4 Response Without an Invitation

If no second human participant needs to join, the `invitation` property is omitted.

Example for a human-to-model interaction:

```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "interactionToken": "interaction-scoped-access-token",
  "status": "waitingForRegistration",
  "model": {
    "id": "social-exchange",
    "version": "2.1.0"
  },
  "numberOfTurns": 20,
  "currentTurn": 0,
  "agents": {
    "A": {
      "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
      "role": "A",
      "type": "human",
      "registrationStatus": "pending"
    },
    "B": {
      "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
      "role": "B",
      "type": "model",
      "registrationStatus": "registered"
    }
  },
  "createdAt": "2026-07-31T08:45:00Z"
}
```


5.5 Invitation Code Requirements

The invitation code:

- Consists of exactly six decimal digits
- Is generated only when a second human participant must join
- Is bound to one specific interaction
- Is bound to one specific human agent
- Can be used only once
- Expires after a configurable period
- Is not permanent authentication
- Is exchanged for the interaction token when the invited participant joins

Its format is:

```
^[0-9]{6}$
```

Example:

```
483921
```

The JavaScript application that creates the interaction should display the invitation code to the initiating participant.

The initiating participant can communicate the code to the second participant through a separate communication channel.

The interaction token and invitation code must not be included in ordinary application logs.

6. Register the Initiating Human Participant

```
POST /api/v1/interactions/{sessionId}/agents/{agentId}/register
Authorization: Bearer <interactionToken>
```

Registers a human participant who already has access to the interaction token.

This will normally be the participant using the JavaScript application instance that created the interaction.

Formal model agents are registered automatically by the server and must not call this endpoint.

6.1 Request

```
{
  "participant": {
    "participantId": "participant-1042",
    "displayName": "Participant A"
  },
  "client": {
    "clientId": "browser-installation-91c5"
  }
}
```

The following properties may be optional depending on the application:

- `participantId`
- `displayName`
- `clientId`

The API should not require personally identifying information unless it is necessary for the research or application context.

6.2 Response

```
200 OK
```

```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
  "agentRole": "A",
  "registrationStatus": "registered",
  "sessionStatus": "waitingForRegistration",
  "registeredAt": "2026-07-31T08:46:00Z"
}
```

For a human-to-human interaction, the session remains in `waitingForRegistration` until the invited participant joins and registers agent `B`.

For a human-to-model interaction, successful registration of the human participant may cause the session to transition directly to `running`.

Registration should be idempotent for the same agent and participant.

Attempting to register a different participant for an already registered agent should return:

```
409 Conflict
```

---

7. Join an Interaction Using an Invitation Code

```
POST /api/v1/interactions/join
```

Allows another instance of the JavaScript application to join an existing interaction by entering the six-digit invitation code.

No bearer token is required because the joining participant does not yet possess the interaction token.

The endpoint performs the following operations:

1. Resolves the interaction associated with the invitation code
2. Resolves the human agent associated with the invitation
3. Registers the joining participant for that agent
4. Invalidates the invitation code
5. Returns the interaction token and assigned agent ID

These operations must be performed atomically.

7.1 Request

```
{
  "invitationCode": "483921",
  "participant": {
    "participantId": "participant-2048",
    "displayName": "Participant B"
  },
  "client": {
    "clientId": "browser-installation-a842"
  }
}
```

7.2 Response

```
200 OK
```

```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "interactionToken": "interaction-scoped-access-token",
  "sessionStatus": "running",
  "agent": {
    "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
    "agentRole": "B",
```

```
    "type": "human",
    "registrationStatus": "registered"
  },
  "model": {
    "id": "social-exchange",
    "version": "2.1.0"
  },
  "numberOfTurns": 20,
  "currentTurn": 0,
  "registeredAt": "2026-07-31T08:47:00Z"
}
```


After a successful join request:

- The invited human agent is registered
- The invitation code becomes invalid
- The joining JavaScript application receives the interaction token
- The joining application stores the returned agent ID
- The interaction starts automatically if all required agents are registered

7.3 Invalid Invitation Code

An unknown or invalid invitation code returns:

```
404 Not Found
```


```
{
  "type": "https://api.example.org/problems/invalid-invitation",
  "title": "Invalid invitation code",
  "status": 404,
  "detail": "The invitation code is invalid or no longer available."
}
```


The response should not disclose whether the code previously existed.

7.4 Expired Invitation Code

```
410 Gone
```


```
{
  "type": "https://api.example.org/problems/invitation-expired",
  "title": "Invitation code expired",
  "status": 410,
  "detail": "The invitation code has expired."
}
```


7.5 Already Used Invitation Code

```
409 Conflict
```

```
{
  "type": "https://api.example.org/problems/invitation-already-used",
  "title": "Invitation code already used",
  "status": 409,
  "detail": "The human agent associated with this invitation has already been registered."
}
```

7.6 Invitation Rate Limiting

Because an invitation code contains only six digits, the join endpoint must be rate-limited.

After too many unsuccessful attempts, the server returns:

```
429 Too Many Requests
```

```
{
  "type": "https://api.example.org/problems/invitation-rate-limit",
  "title": "Too many invitation attempts",
  "status": 429,
  "detail": "Too many unsuccessful invitation attempts were submitted.",
  "retryAfterSeconds": 60
}
```

Rate limiting should consider:

- Source IP address
- Browser or client identifier, where available
- Failed attempts per invitation code
- Failed attempts across all invitation codes

---

8. Retrieve Interaction Status

```
GET /api/v1/interactions/{sessionId}
Authorization: Bearer <interactionToken>
```

Returns the current interaction state.

The response may include the status of an invitation, but it must not return the invitation code again.

8.1 Response

```
200 OK
```

```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "status": "waitingForRegistration",
  "model": {
    "id": "social-exchange",
```

```
    "version": "2.1.0"
  },
  "numberOfTurns": 20,
  "currentTurn": 0,
  "activeAgentIds": [],
  "agents": {
    "A": {
      "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
      "role": "A",
      "type": "human",
      "registrationStatus": "registered"
    },
    "B": {
      "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
      "role": "B",
      "type": "human",
      "registrationStatus": "pending"
    }
  },
  "invitation": {
    "agentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
    "agentRole": "B",
    "status": "pending",
    "expiresAt": "2026-07-31T09:15:00Z"
  },
  "persistence": {
    "enabled": true
  },
  "createdAt": "2026-07-31T08:45:00Z",
  "startedAt": null
}
```


Possible invitation statuses are:

```
pending
used
expired
revoked
```

When the interaction is running, `activeAgentIds` identifies the agents from which an action is currently expected.

Using an array allows the protocol to support:

- Alternating interactions
- Simultaneous interactions
- Models in which no human action is currently required

---

9. Poll for a Perception

```
GET /api/v1/interactions/{sessionId}/agents/{agentId}/perception
Authorization: Bearer <interactionToken>
```

Returns the current perception state for the specified agent.

The endpoint supports regular polling.

Optional query parameters may be used for long polling:

```
after=<cursor>
wait=<seconds>
```

Example:

```
GET /api/v1/interactions/{sessionId}/agents/{agentId}/perception?after=18&wait=25
```

The `wait` value specifies how long the server may keep the request open while waiting for a new perception.

The server may enforce a maximum waiting time.

9.1 Pending Response

No new perception is currently available.

```
200 OK
```

```
{
  "state": "pending",
  "sessionStatus": "running",
  "currentTurn": 7,
  "cursor": "18",
  "retryAfterMilliseconds": 1000
}
```

9.2 Perceivable Response

A perception is available for the agent.

```
200 OK
```

```
{
  "state": "perceivable",
  "sessionStatus": "running",
  "perceptionId": "349fbc68-d9ca-46de-84bb-f0b55306588e",
  "cursor": "19",
  "turn": 8,
  "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
  "actionRequired": true,
  "experience": {
    "$comment": "Example value conforming to the model's stateSpaceSchema."
  },
  "createdAt": "2026-07-31T08:47:12Z"
}
```

The complete `experience` object must conform to the model's shared `stateSpaceSchema`.

9.3 Interaction-Ended Response

```
200 OK
```

```
{
  "state": "interactionEnded",
  "sessionStatus": "completed",
  "finalTurn": 20,
  "endReason": "numberOfTurnsReached",
  "endedAt": "2026-07-31T08:55:41Z"
}
```

The perception endpoint must satisfy the following requirements:

- A `GET` request must not consume or delete a perception
- Repeating the same request must be safe
- The cursor must allow clients to detect already processed perceptions
- A client must only request perceptions for its assigned agent
- The server remains authoritative regarding whether an action is required

---

10. Submit Behavior

```
POST /api/v1/interactions/{sessionId}/agents/{agentId}/actions
Authorization: Bearer <interactionToken>
```

Submits the behavior of an agent for the specified interaction turn.

10.1 Request

```
{
  "actionId": "client-generated-action-uuid",
  "turn": 8,
  "behavior": {
    "$comment": "Example value conforming to the model's stateSpaceSchema."
  }
}
```

The complete `behavior` object must conform to the same `stateSpaceSchema` used for the perception's `experience` object.

```
schema(experience) = schema(behavior)
```

10.2 Response

```
202 Accepted
```

```
{
  "status": "accepted",
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
  "actionId": "client-generated-action-uuid",
  "turn": 8,
  "sessionStatus": "running",
  "acceptedAt": "2026-07-31T08:47:20Z"
}
```

10.3 Idempotency

`actionId` is a client-generated idempotency identifier.

Repeating the same action with the same `actionId` must not advance the interaction more than once.

If the same action is submitted again with identical content, the server should return the original acceptance response.

If an existing `actionId` is reused with different content, the server must return:

```
409 Conflict
```

10.4 Action Conflicts

The server should return `409 Conflict` when:

- The interaction has already ended
- The submitted turn is no longer current
- The agent is not expected to act
- An action has already been accepted for that agent and turn
- An existing `actionId` is reused with different content

The server should return `422 Unprocessable Content` when the submitted `behavior` does not conform to the model's `stateSpaceSchema`.

---

11. End an Interaction

```
POST /api/v1/interactions/{sessionId}/end
Authorization: Bearer <interactionToken>
```

Terminates the complete interaction before its normal completion.

11.1 Request

```
{
  "reason": "terminatedByClient"
}
```

11.2 Response

```
200 OK
```

```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "status": "terminated",
  "endReason": "terminatedByClient",
  "finalTurn": 8,
  "endedAt": "2026-07-31T08:48:00Z"
}
```

The operation should be idempotent.

Repeating the request for an already terminated interaction should return the existing termination result.

A naturally completed interaction does not need to be explicitly terminated.

Possible end reasons include:

```
numberOfTurnsReached
terminatedByClient
participantDisconnected
registrationTimeout
interactionExpired
modelFailure
serverError
```

---

12. Persistence

When persistence is enabled, the server may store:

- Interaction configuration
- Model ID and model version
- Agent configuration
- Participant registrations
- Submitted behaviors
- Generated perceptions
- Turn transitions
- Interaction state transitions
- Timestamps
- Completion or termination reasons

When persistence is disabled:

- Data should be retained only as long as operationally necessary
- Retrieval after the interaction has ended should not be guaranteed
- Temporary operational logging should be documented separately
- Personally identifying information should not be stored unnecessarily

Persisted results can be exposed through an optional results endpoint.

```
GET /api/v1/interactions/{sessionId}/results
Authorization: Bearer <interactionToken>
```

12.1 Response

```
200 OK
```


```
{
  "sessionId": "2fd2bfb0-f193-4f34-a16d-a1a5936bc734",
  "status": "completed",
  "model": {
    "id": "social-exchange",
    "version": "2.1.0"
  },
  "numberOfTurns": 20,
  "turns": [
    {
      "turn": 1,
      "actorAgentId": "574dcfd4-cf16-42b7-9611-95b2356c4bdc",
      "behavior": {
        "$comment": "Value conforming to stateSpaceSchema."
      },
      "perceptions": [
        {
          "agentId": "e388a210-a77a-4aee-9b40-da115d3e31aa",
          "experience": {
            "$comment": "Value conforming to the same stateSpaceSchema."
          }
        }
      ],
      "createdAt": "2026-07-31T08:45:12Z"
    }
  ],
  "completedAt": "2026-07-31T08:55:41Z"
}
```


If persistence was disabled, the server may return:

```
404 Not Found
```

or:

```
410 Gone
```

depending on whether the interaction previously existed but its data has already been removed.

13. Error Format

Errors use the `application/problem+json` media type.

13.1 Example

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
```

```
{
  "type": "https://api.example.org/problems/invalid-state-vector",
  "title": "Invalid state vector",
  "status": 422,
  "detail": "The submitted behavior does not conform to the model's state-space schema.",
  "instance": "/api/v1/interactions/2fd2bfb0-f193-4f34-a16d-a1a5936bc734/agents/e388a210-
      a77a-4aee-9b40-da115d3e31aa/actions",
  "errors": [
    {
      "path": "/behavior/exampleProperty",
      "code": "maximum",
      "message": "The value exceeds the permitted maximum."
    }
  ],
  "requestId": "req-7aa93f29"
}
```

13.2 Recommended HTTP Status Codes

Status	Meaning
`400 Bad Request`	The request is malformed
`401 Unauthorized`	The interaction token is missing or invalid
`403 Forbidden`	The requested operation is not permitted
`404 Not Found`	The model, interaction, agent, or invitation does not exist
`409 Conflict`	The request conflicts with the interaction, registration, or turn state
`410 Gone`	The interaction or invitation has expired
`422 Unprocessable Content`	The state vector or configuration violates the applicable schema
`429 Too Many Requests`	The client exceeded the polling, action, or invitation attempt limit
`500 Internal Server Error`	An unexpected server error occurred

14. Endpoint Summary

Method	Endpoint	Purpose
GET	`/api/v1/models`	List available simulation models
GET	`/api/v1/models/{modelId}`	Retrieve model metadata and the shared state-space schema
POST	`/api/v1/interactions`	Create an interaction and, where required, generate an invitation code
POST	`/api/v1/interactions/join`	Join an interaction and register the invited human participant
GET	`/api/v1/interactions/{sessionId}`	Retrieve the current interaction status
POST	`/api/v1/interactions/{sessionId}/agents/{agentId}/register`	Register a human participant who already has the interaction token
GET	`/api/v1/interactions/{sessionId}/agents/{agentId}/perception`	Poll for an agent's perception
POST	`/api/v1/interactions/{sessionId}/agents/{agentId}/actions`	Submit an agent's behavior
POST	`/api/v1/interactions/{sessionId}/end`	Terminate the interaction
GET	`/api/v1/interactions/{sessionId}/results`	Retrieve persisted interaction results

---

15. Core Implementation Requirements

The server must enforce the following requirements:

1. Every interaction is bound to an exact model ID and model version.
2. Every model exposes one shared `stateSpaceSchema`.
3. Perceptual experience vectors and behavioral vectors use the same schema.
4. Every returned experience vector is valid according to the shared schema.
5. Every submitted behavioral vector is validated against the shared schema.
6. Human agent slots must be registered before the corresponding participant can interact.
7. Formal model agents are registered and executed internally by the server.
8. The server starts an interaction only after all required human participants have registered.
9. The server is the authoritative source for interaction states and turn numbers.
10. Each required agent can submit no more than one behavior per turn.
11. Action submission is idempotent.
12. Polling does not consume perceptions.
13. Interaction state transitions are atomic.

14. The interaction token is scoped to one interaction.
15. The invitation code contains exactly six decimal digits.
16. The invitation code is generated using a cryptographically secure random-number generator.
17. The invitation code is unique among all active invitations.
18. The invitation code is bound to exactly one interaction and one human agent.
19. The invitation code cannot be used to register a different agent.
20. The invitation code can be used only once.
21. Successful invitation redemption and participant registration occur atomically.
22. Expired, revoked, or used invitation codes cannot be reused.
23. Concurrent join requests cannot register multiple participants for the same agent.
24. The invitation join endpoint is rate-limited.
25. Invitation codes and interaction tokens are never written to ordinary application logs.
26. Invitation codes should not be stored in plaintext when avoidable.
27. Invitation codes are not accepted as authentication after registration.
28. The joining application receives the interaction token only after successful registration.
29. Interaction creation and invitation-code generation occur in one atomic operation.
30. All communication takes place over HTTPS.
31. CORS access is restricted to approved JavaScript application origins.
32. Persistence behavior is clearly defined for both persistent and non-persistent interactions.